



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 6

Integrante(s): 120002850 323064039

323071378 323214636

426059576 426058744

Semestre: 2027-1

Fecha de entrega: 9 de octubre de 2026

Observaciones:

CALIFICACIÓN:

Índice

1. Introducción	4
1.1. Planteamiento del problema	4
1.2. Motivación	4
1.3. Objetivos	4
2. Marco Teórico	4
2.1. Archivos JAR y portabilidad	4
2.2. Documentación con Javadoc	5
2.3. Paquetes y estructura de directorios	5
2.4. Comandos básicos para crear y correr un JAR en consola	5
2.5. Cómo se compilan y cómo se corren los archivos Java empaquetados	5
3. Desarrollo	6
3.1. Investigación con inteligencia artificial	6
3.2. Organización recopilatoria por paquetes	9
3.3. Diagramas UML y de flujo estandarizados	9
3.4. Recopilatorio por clase	10
3.4.1. Suma de P1 (Calculadora y operaciones basicas)	10
3.4.2. Resta de P1 (Calculadora y operaciones basicas)	10
3.4.3. Multiplicacion de P1 (Calculadora y operaciones basicas)	10
3.4.4. Division de P1 (Calculadora y operaciones basicas)	10
3.4.5. Exponente de P1 (Calculadora y operaciones basicas)	14
3.4.6. Raiz de P1 (Calculadora y operaciones basicas)	14
3.4.7. Calculadora de P1 (Calculadora y operaciones basicas)	14
3.4.8. Figura de P1 (Editor grafico de figuras)	14
3.4.9. CanvasPanel de P1 (Editor grafico de figuras)	19
3.4.10. EditorFrame de P1 (Editor grafico de figuras)	19
3.4.11. Ejercicio1_P2 de P2 (Estructuras de control y tablas)	19
3.4.12. Ejercicio2_P2 de P2 (Estructuras de control y tablas)	19
3.4.13. Ejercicio3_P2 de P2 (Estructuras de control y tablas)	22
3.4.14. Palindromo de P3 (Palindromos y colecciones)	22
3.4.15. AnalizadorPalabras de P4 (Análisis de palabras)	22
3.4.16. Main de P4 (Análisis de palabras)	27
3.4.17. Empleado de P5 (Encapsulamiento empleado y fecha)	27
3.4.18. Fecha de P5 (Encapsulamiento empleado y fecha)	27
3.4.19. MainEmpleado de P5 (Encapsulamiento empleado y fecha)	27
3.4.20. MainFecha de P5 (Encapsulamiento empleado y fecha)	27
3.4.21. Medico de Proyecto (Sistema hospitalario)	32
3.4.22. Enfermero de Proyecto (Sistema hospitalario)	32
3.4.23. Paciente de Proyecto (Sistema hospitalario)	32
3.4.24. Sistema de Proyecto (Sistema hospitalario)	32
3.4.25. Main de Proyecto (Sistema hospitalario)	37

4. Resultados	37
4.1. Compilación de archivos empaquetados	37
4.2. Ejecución por classpath	37
4.3. Creación del JAR y verificación de contenido	39
4.4. Ejecución del JAR	39
5. Conclusiones	39

1. Introducción

1.1. Planteamiento del problema

En el curso de programación orientada a objetos se acumularon cinco prácticas y un primer avance del proyecto hospitalario con decenas de clases dispersas en distintos estilos de organización. Sin un esquema común de paquetes resulta difícil distribuir los programas, evitar colisiones de nombres y reutilizar los binarios en diferentes equipos. Además, la ausencia de documentación uniforme limita la comprensión de cada módulo. Esta práctica aborda el problema mediante la recopilación de todas las entregas bajo la jerarquía institucional, la compilación y ejecución de archivos empaquetados y la generación de archivos JAR ejecutables con su documentación correspondiente.

1.2. Motivación

La industria del software distribuye componentes como unidades portables y autodocumentadas para reducir errores de integración y facilitar el mantenimiento [1]. El uso de paquetes con dominio inverso garantiza trazabilidad académica y separa espacios de nombres por práctica y proyecto [3]. El empaquetado en un único archivo con manifiesto ejecutable simplifica el despliegue sin manipular clases sueltas, mientras que la documentación generada con la herramienta oficial convierte comentarios estructurados en manuales navegables [2]. Reunir las cinco prácticas y el proyecto en un recopilatorio homogéneo permite verificar que la compilación, la ejecución por classpath y la ejecución por JAR producen el mismo comportamiento.

1.3. Objetivos

- El objetivo principal consiste en recopilar las cinco prácticas y el primer avance del proyecto bajo la jerarquía `mx.unam.fi.die.poo.g7`.
- Organizar la estructura común de directorios.
- Aprender cómo se compilan los archivos empaquetados.
- Entender cómo se crea un JAR y cómo se ejecuta.
- De forma complementaria se debe documentar cada clase con Javadoc independiente, se estandarizan los diagramas UML y de flujo de todas las clases.

2. Marco Teórico

2.1. Archivos JAR y portabilidad

Un archivo JAR es un contenedor basado en ZIP que agrupa clases compiladas, recursos y metadatos en una sola unidad transportable [3]. El manifiesto `MANIFEST.MF`

con la directiva **Main-Class** indica la clase de entrada para la ejecución directa. Esta técnica preserva la jerarquía de paquetes, reduce transferencias y evita pérdidas de archivos sueltos.

2.2. Documentación con Javadoc

Javadoc transforma comentarios `/** ... */` en páginas HTML navegables [2]. Cada clase, atributo, constructor y método se describe en lenguaje natural con **@param** para argumentos, **@return** para resultados y **@throws** para excepciones reales. La generación determinista desde consola con codificación UTF-8 y classpath controlado produce manuales reproducibles por clase sin enlaces entre clases.

2.3. Paquetes y estructura de directorios

Los paquetes con dominio inverso en minúsculas evitan homónimos y delimitan módulos [1]. El prefijo `mx.unam.fi.die.poo.g7` identifica a la Facultad de Ingeniería y se subdivide en `p1` a `p5` y `proyecto`. La disposición ubica fuentes en `src/`, binarios en `test/`, el ejecutable en `lib/` y la documentación en `doc/`, lo que separa editables de productos y facilita el versionado.

2.4. Comandos básicos para crear y correr un JAR en consola

En Windows (`cmd`) y en Linux la herramienta `jar` del JDK crea y lista archives con sintaxis equivalente. Para crear un JAR con manifiesto ejecutable se emplea `jar cfm lib/SistemaHospitalario.jar MANIFEST.MF -C test mx/unam/fi/die/poo/g7/proyecto`, donde `c` crea, `f` indica archivo, `m` incorpora manifiesto y `-C` cambia al directorio de clases para preservar la jerarquía. El contenido se verifica con `jar tf lib/SistemaHospitalario.jar`, que lista entradas y manifiesto. La ejecución directa usa `java -jar lib/SistemaHospitalario.jar`, que localiza **Main-Class** sin declarar classpath. Para ejecutar una clase concreta sin manifiesto se usa `java -cp lib/SistemaHospitalario.jar mx.unam.fi.die.poo.g7.proyecto.Main`. En `cmd` las rutas usan diagonal invertida y las variables de entorno se ajustan con `set`, pero los subcomandos de `jar` y `java` conservan el mismo orden.

2.5. Cómo se compilan y cómo se corren los archivos Java empaquetados

Un archivo con declaración `package` debe compilarse con destino explícito para reproducir la jerarquía: `javac -encoding UTF-8 -parameters -d test src/mx/unam/fi/die/poo/g7/` genera `test/mx/.../*.class` con la codificación y los metadatos de parámetros requeridos. La compilación no produce ejecutable, solo bytecode verificable por la máquina virtual. La ejecución por classpath indica dónde buscar clases: `java -cp test mx.unam.fi.die.poo.g7.proyecto.Main` resuelve el nombre totalmente calificado contra `test` y arranca el método de entrada con la entrada estándar como única fuente determinista. Cuando las clases ya están empaquetadas, el classpath puede ser el JAR o el JAR puede autoarrancar con `-jar` si el manifiesto declara la clase

principal. Ambos modos deben producir la misma salida ante la misma entrada, lo que demuestra portabilidad del empaquetado.

3. Desarrollo

3.1. Investigación con inteligencia artificial

Se formuló una consulta común de 90 palabras conservada en `doc/Capturas/query.txt` sobre mejores prácticas de JAR, generación de Javadoc y estructura `src`, `test` y `doc`. Cada integrante la planteó a un modelo de lenguaje para recabar criterios profesionales.

En la figura 3.1 se reserva la evidencia de Alan. En la figura 3.2 se reserva la

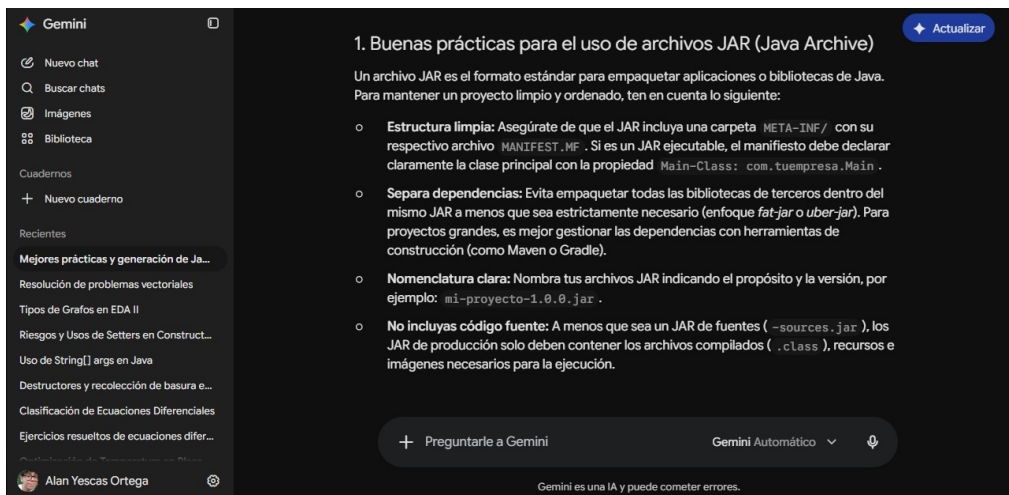


Figura 3.1: Consulta de investigación sobre JAR y Javadoc por Alan.

evidencia de Zoé. En la figura 3.3 se reserva la evidencia de Luis. En la figura 3.4 se reserva la evidencia de Joseph. En la figura 3.5 se reserva la evidencia de Hanniel.

Discusión. Al realizar una comparación de cada una de las respuestas llegamos a la conclusión de que para trabajar de forma profesional con Java, es fundamental mantener un orden riguroso: los archivos JAR deben estar correctamente organizados y tener un manifiesto bien configurado. Por su parte, el Javadoc debe enfocarse en explicar qué hace cada método y cómo usarlo, usando etiquetas estándar como `@param` y `@return` en lugar de describir detalles técnicos internos. La mejor manera de trabajar es automatizar siempre la generación de la documentación a través de estas herramientas de construcción, asegurando así que el código y su manual técnico estén siempre actualizados y listos para ser utilizados por otros desarrolladores.

La síntesis confirma que la mejor manera de generar Javadoc es invocar la herramienta oficial desde consola con UTF-8 en entrada, salida y charset, con classpath compilado y sourcepath vacío, una clase por invocación y salida aislada por clase. El JAR debe contener solo clases verificadas con manifiesto preciso. La estructura

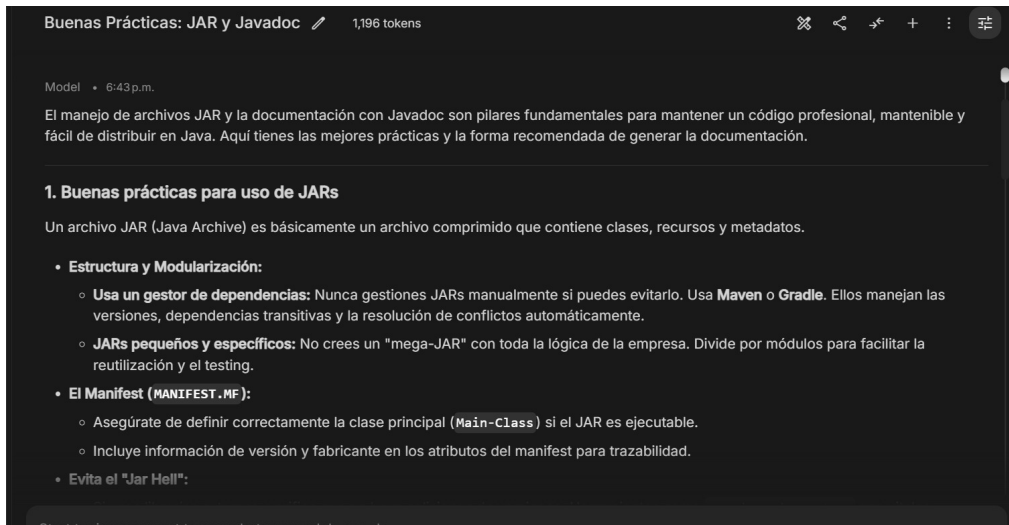


Figura 3.2: Consulta de investigación sobre JAR y Javadoc por Zoé.

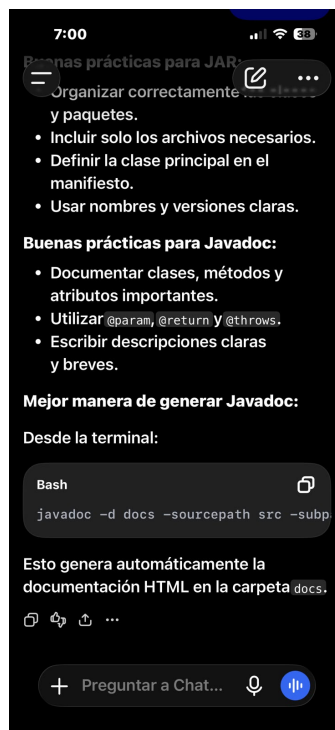


Figura 3.3: Consulta de investigación sobre JAR y Javadoc por Luis.

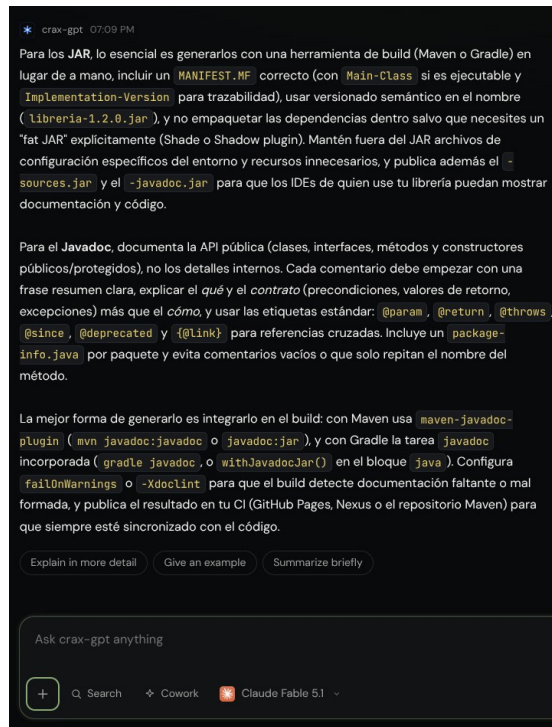


Figura 3.4: Consulta de investigación sobre JAR y Javadoc por Joseph.

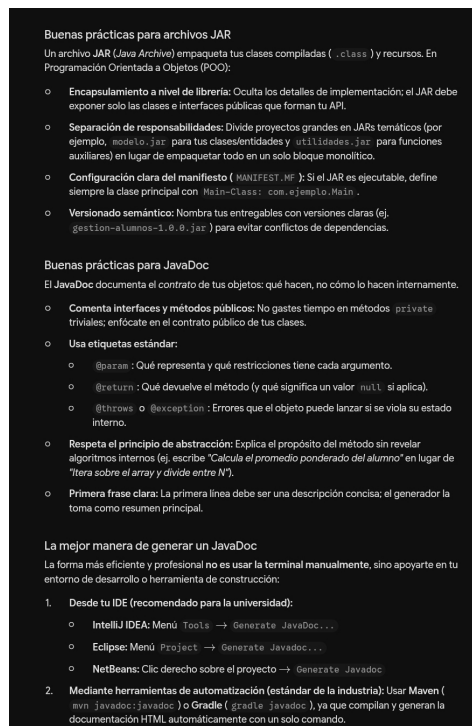


Figura 3.5: Consulta de investigación sobre JAR y Javadoc por Hannel.

homogénea separa fuentes, binarios, biblioteca y documentación para garantizar mantenibilidad y portabilidad.

3.2. Organización recopilatoria por paquetes

Se reunieron las entregas sin crear código nuevo, solo copiándolas bajo la jerarquía institucional:

- `mx.unam.fi.die.poo.g7.p1`: calculadora básica con siete operaciones y editor gráfico con figuras, lienzo y ventana.
- `mx.unam.fi.die.poo.g7.p2`: conteo de aprobados, mayor de diez números y tabla de múltiplos.
- `mx.unam.fi.die.poo.g7.p3`: lectura validada de entero de cinco dígitos y verificación de palíndromo.
- `mx.unam.fi.die.poo.g7.p4`: análisis de frecuencias de palabras con colecciones y programa coordinador.
- `mx.unam.fi.die.poo.g7.p5`: entidades empleado y fecha con encapsulamiento y programas de ejemplo.
- `mx.unam.fi.die.poo.g7.proyecto`: sistema hospitalario con médico, enfermero, paciente, contenedor y menú principal.

Cada módulo conserva su lógica original y solo se añadieron comentarios Javadoc y constructores explícitos donde faltaban para documentar el implícito. La colisión de nombre simple entre las dos clases `Main` de `p4` y `proyecto` se resolvió organizando Javadoc, diagramas y pseudocódigos por carpetas de origen (`P4/` y `Proyecto/`), sin renombrar clases ni inventar sufijos en el código. Las relaciones se explican por paquetes: `AnalizadorPalabras` es usado por `Main` de `p4`; `Empleado` y `Fecha` son usados por sus programas principales; `Sistema` contiene listas de `Medico`, `Enfermero` y `Paciente`; el editor relaciona `Figura`, `CanvasPanel` y `EditorFrame` por composición y uso del contexto gráfico.

3.3. Diagramas UML y de flujo estandarizados

Por cada clase se generó un UML en tabla blanca con nombre, atributos tipados con visibilidad y operaciones con parámetros nombrados y retorno, con constructores explícitos e implícitos documentados, estáticos subrayados y finales con `{readOnly}`. Por cada clase se generó un flujo de la operación relevante con estilos uniformes, decisiones binarias Sí/No y ramas etiquetadas. Los pseudocódigos reproducen la misma operación con pasos, condiciones y ciclos legibles.

3.4. Recopilatorio por clase

3.4.1. Suma de P1 (Calculadora y operaciones basicas)

Lee dos enteros y muestra la suma. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.6 se muestra el UML de **Suma** y en la figura 3.7 su flujo.

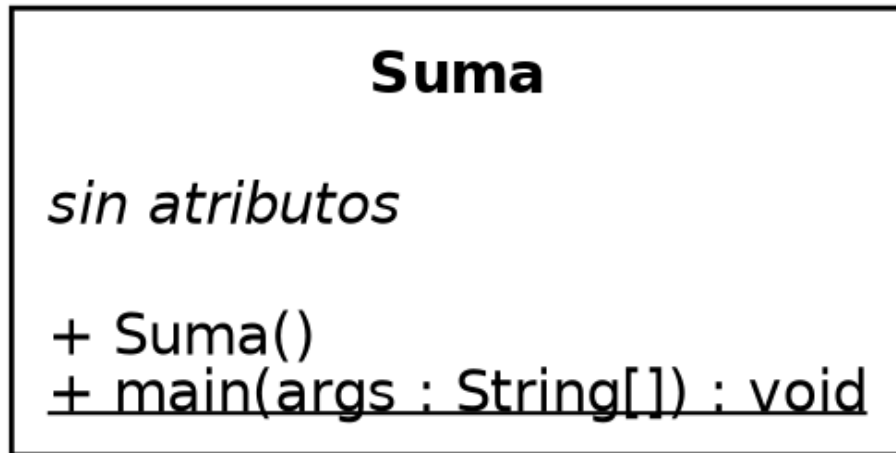


Figura 3.6: Diagrama UML de Suma (P1).

3.4.2. Resta de P1 (Calculadora y operaciones basicas)

Lee dos enteros y opcion y muestra la resta. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.8 se muestra el UML de **Resta** y en la figura 3.9 su flujo.

3.4.3. Multiplicacion de P1 (Calculadora y operaciones basicas)

Lee dos enteros y muestra el producto. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.10 se muestra el UML de **Multiplicacion** y en la figura 3.11 su flujo.

3.4.4. Division de P1 (Calculadora y operaciones basicas)

Divide dos decimales con control de cero. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.12 se muestra el UML de **Division** y en la figura 3.13 su flujo.

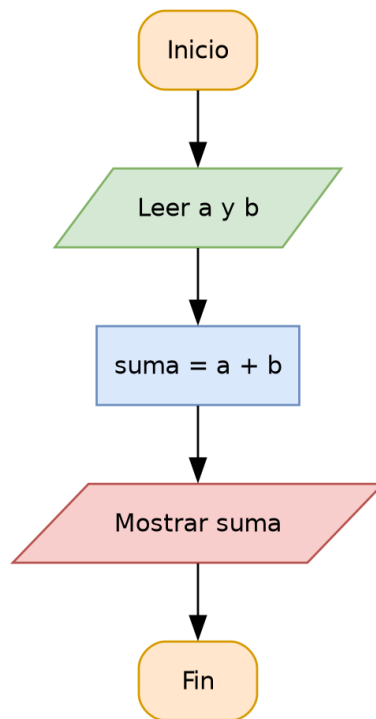


Figura 3.7: Diagrama de flujo de Suma (P1).

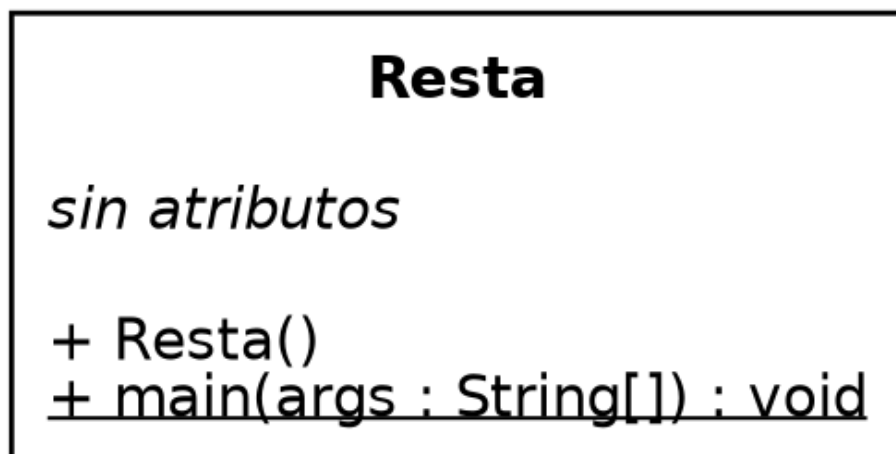


Figura 3.8: Diagrama UML de Resta (P1).

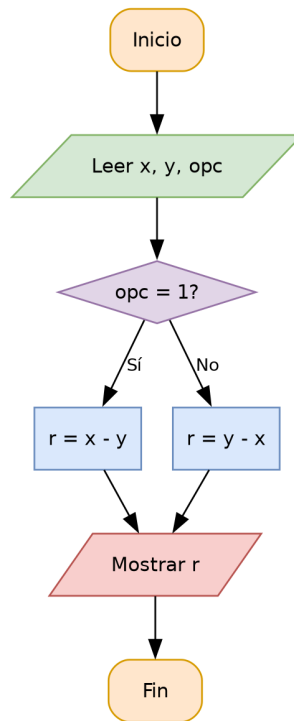


Figura 3.9: Diagrama de flujo de Resta (P1).

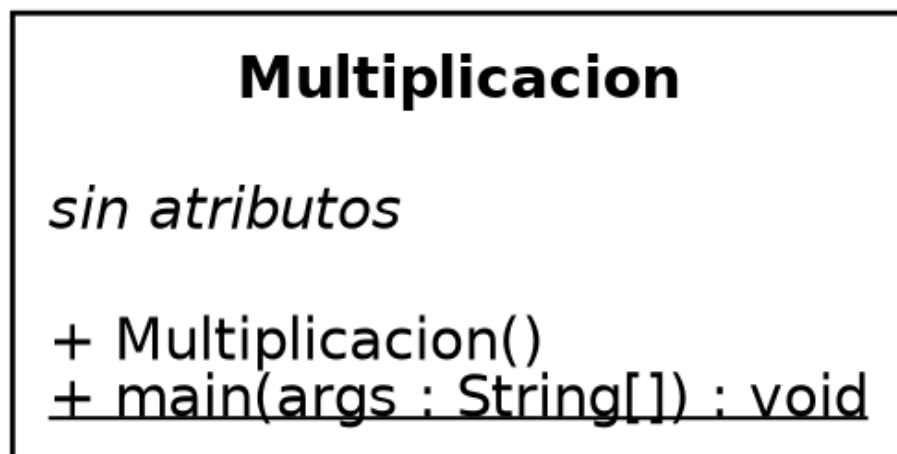


Figura 3.10: Diagrama UML de Multiplicacion (P1).

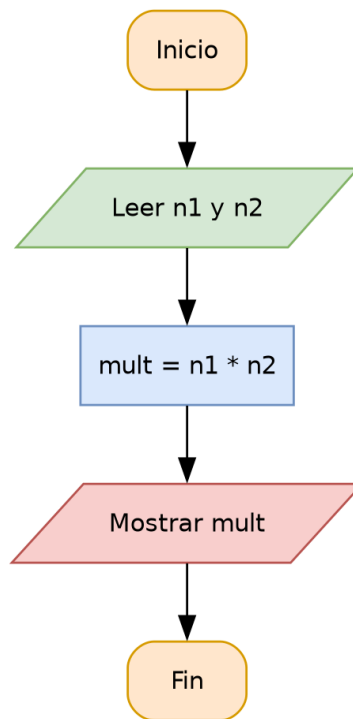


Figura 3.11: Diagrama de flujo de Multiplicacion (P1).

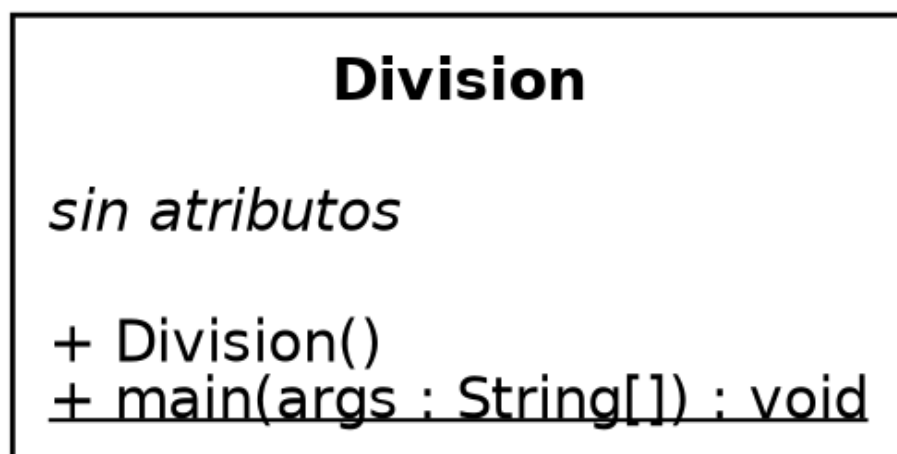


Figura 3.12: Diagrama UML de Division (P1).

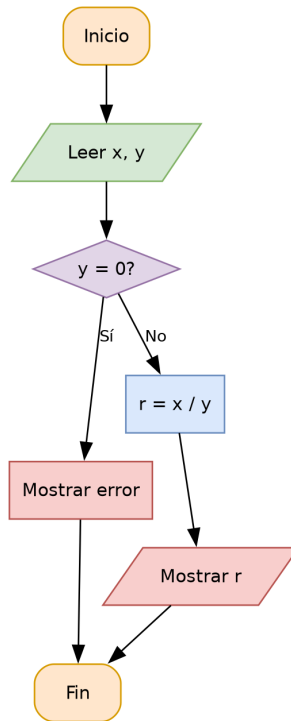


Figura 3.13: Diagrama de flujo de Division (P1).

3.4.5. Exponente de P1 (Calculadora y operaciones basicas)

Calcula potencia con ciclo. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.14 se muestra el UML de **Exponente** y en la figura 3.15 su flujo.

3.4.6. Raiz de P1 (Calculadora y operaciones basicas)

Calcula raiz con control de casos. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.16 se muestra el UML de **Raiz** y en la figura 3.17 su flujo.

3.4.7. Calculadora de P1 (Calculadora y operaciones basicas)

Menu con siete operaciones. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.18 se muestra el UML de **Calculadora** y en la figura 3.19 su flujo.

3.4.8. Figura de P1 (Editor grafico de figuras)

Registro inmutable de figura geometrica. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.20 se muestra el UML de **Figura** y en la figura 3.21 su flujo.

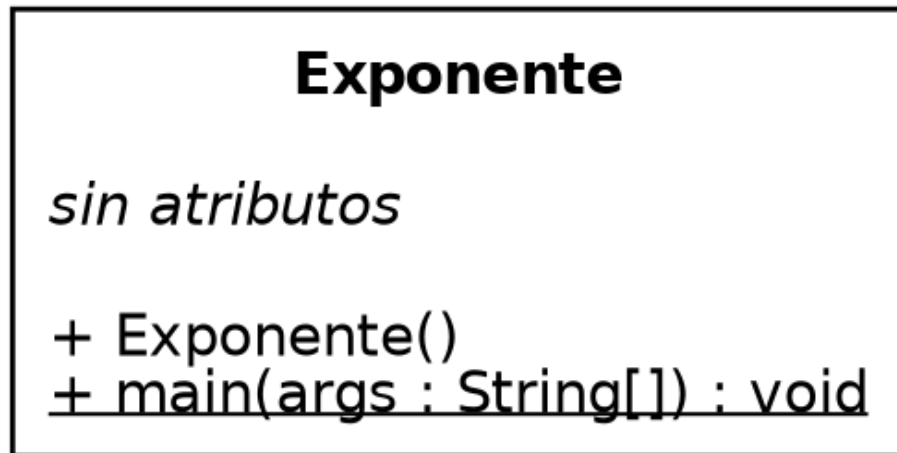


Figura 3.14: Diagrama UML de Exponente (P1).

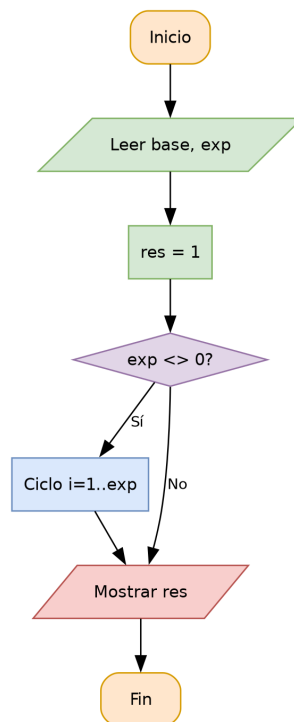


Figura 3.15: Diagrama de flujo de Exponente (P1).

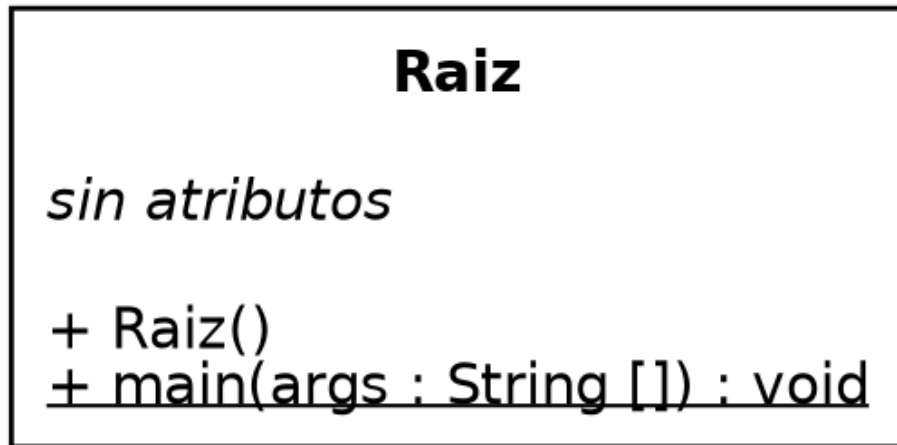


Figura 3.16: Diagrama UML de Raiz (P1).

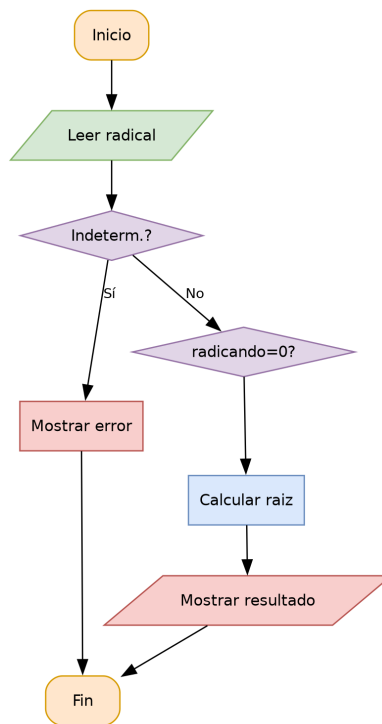


Figura 3.17: Diagrama de flujo de Raiz (P1).

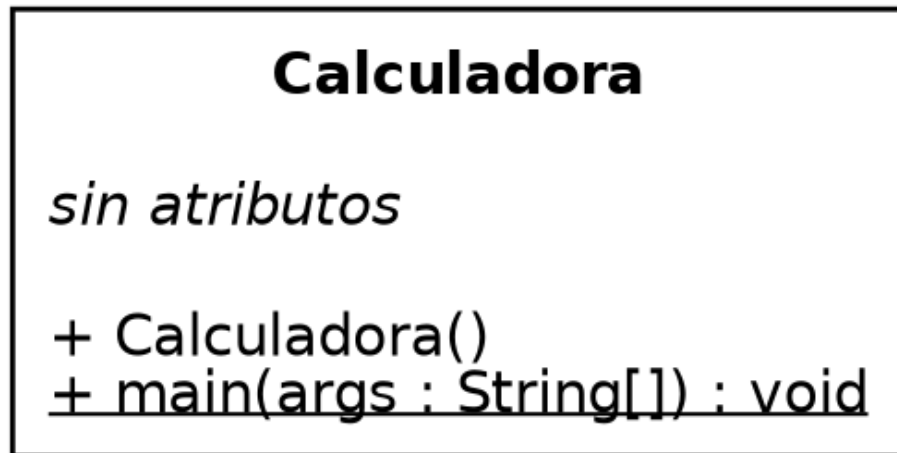


Figura 3.18: Diagrama UML de Calculadora (P1).

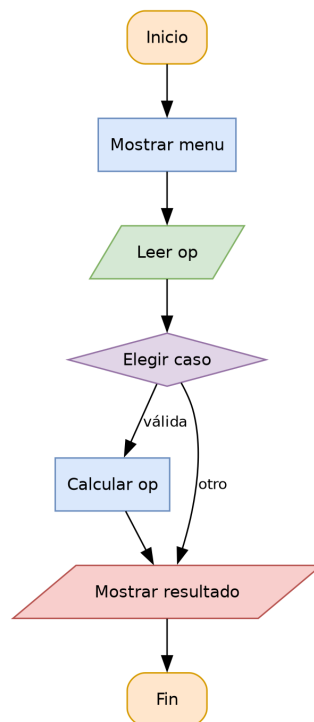


Figura 3.19: Diagrama de flujo de Calculadora (P1).

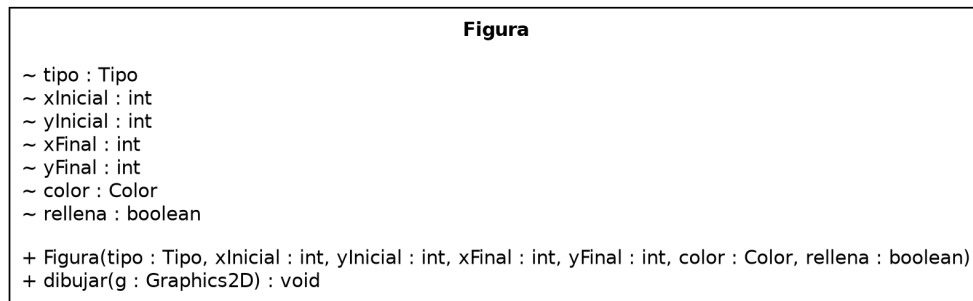


Figura 3.20: Diagrama UML de Figura (P1).

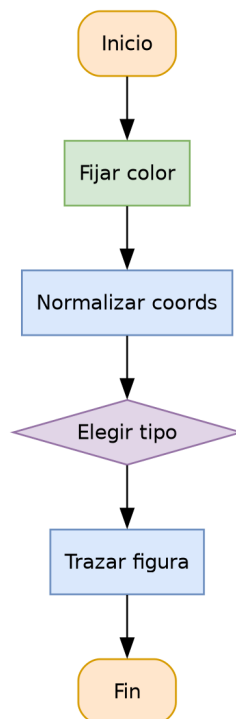


Figura 3.21: Diagrama de flujo de Figura (P1).

3.4.9. CanvasPanel de P1 (Editor grafico de figuras)

Panel de dibujo con deshacer y exportacion. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.22 se muestra el UML de CanvasPanel y en la figura 3.23 su flujo.

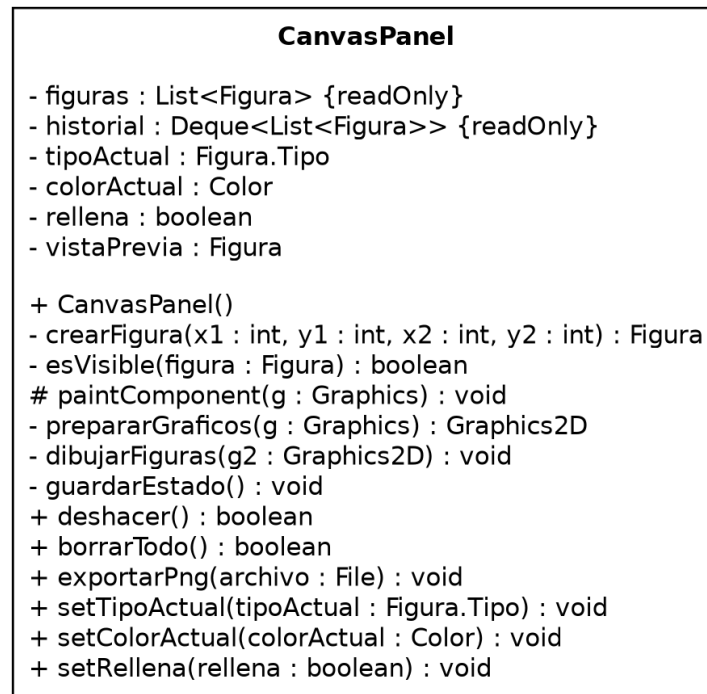


Figura 3.22: Diagrama UML de CanvasPanel (P1).

3.4.10. EditorFrame de P1 (Editor grafico de figuras)

Ventana principal del editor. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.24 se muestra el UML de EditorFrame y en la figura 3.25 su flujo.

3.4.11. Ejercicio1_P2 de P2 (Estructuras de control y tablas)

Cuenta aprobados y reprobados. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.26 se muestra el UML de Ejercicio1_P2 y en la figura 3.27 su flujo.

3.4.12. Ejercicio2_P2 de P2 (Estructuras de control y tablas)

Determina el mayor de diez numeros. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

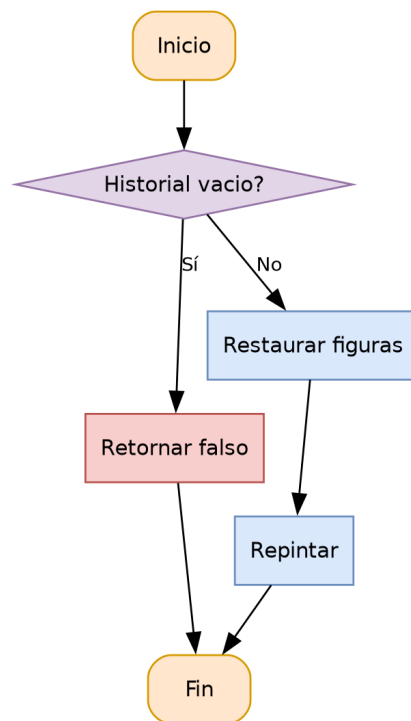


Figura 3.23: Diagrama de flujo de CanvasPanel (P1).

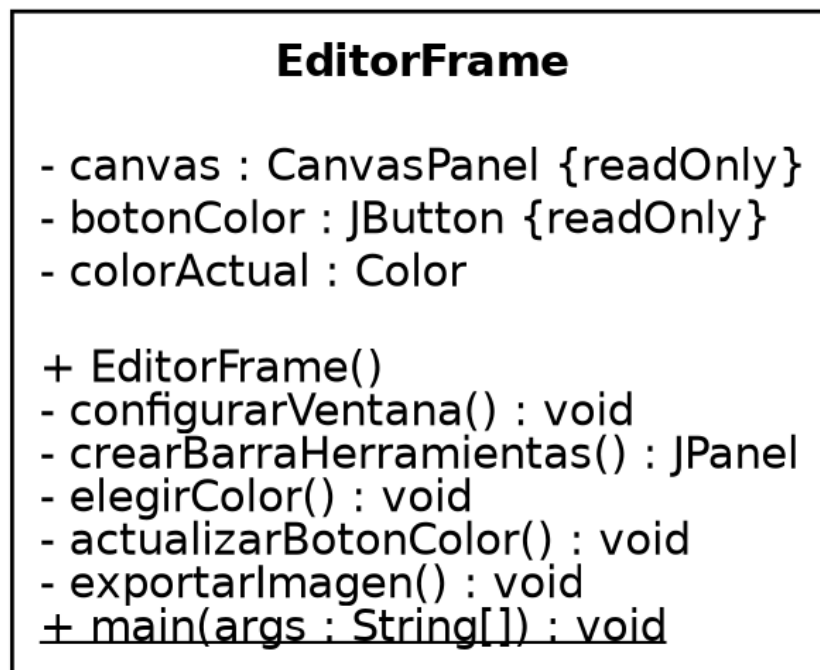


Figura 3.24: Diagrama UML de EditorFrame (P1).

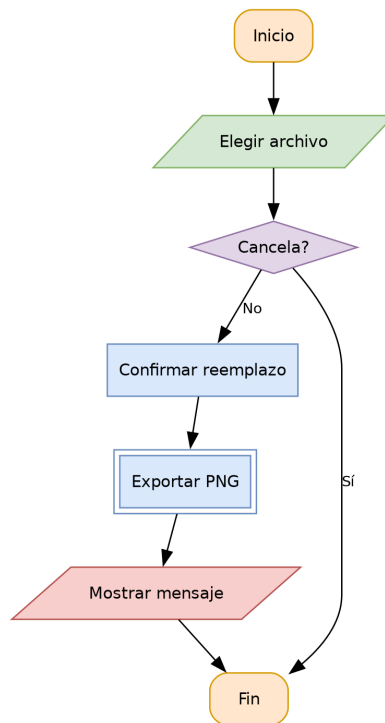


Figura 3.25: Diagrama de flujo de EditorFrame (P1).

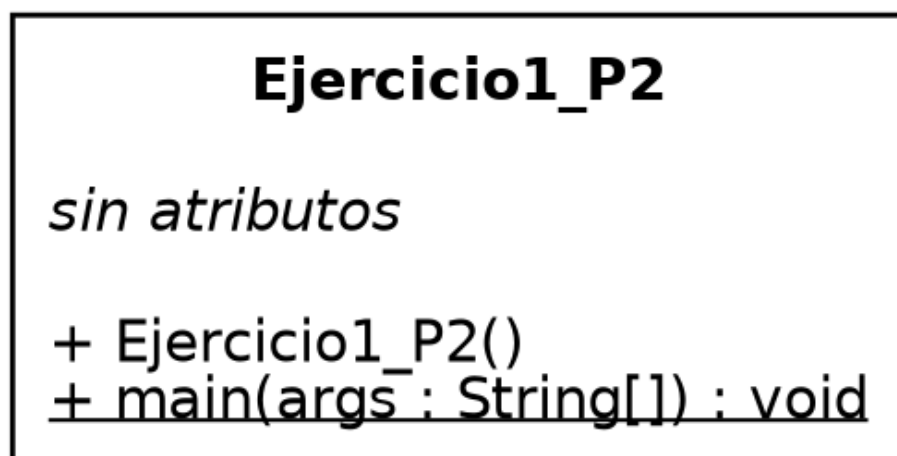


Figura 3.26: Diagrama UML de Ejercicio1_P2 (P2).

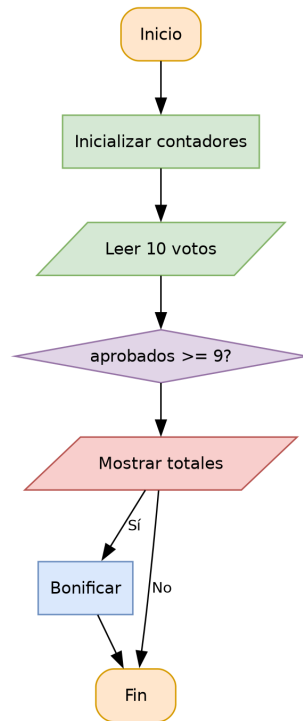


Figura 3.27: Diagrama de flujo de Ejercicio1_P2 (P2).

En la figura 3.28 se muestra el UML de Ejercicio2_P2 y en la figura 3.29 su flujo.

3.4.13. Ejercicio3_P2 de P2 (Estructuras de control y tablas)

Imprime tabla de multiplos. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.30 se muestra el UML de Ejercicio3_P2 y en la figura 3.31 su flujo.

3.4.14. Palindromo de P3 (Palindromos y colecciones)

Verifica palindromo de cinco digitos. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.32 se muestra el UML de Palindromo y en la figura 3.33 su flujo.

3.4.15. AnalizadorPalabras de P4 (Análisis de palabras)

Cuenta frecuencias con colecciones. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.34 se muestra el UML de AnalizadorPalabras y en la figura 3.35 su flujo.

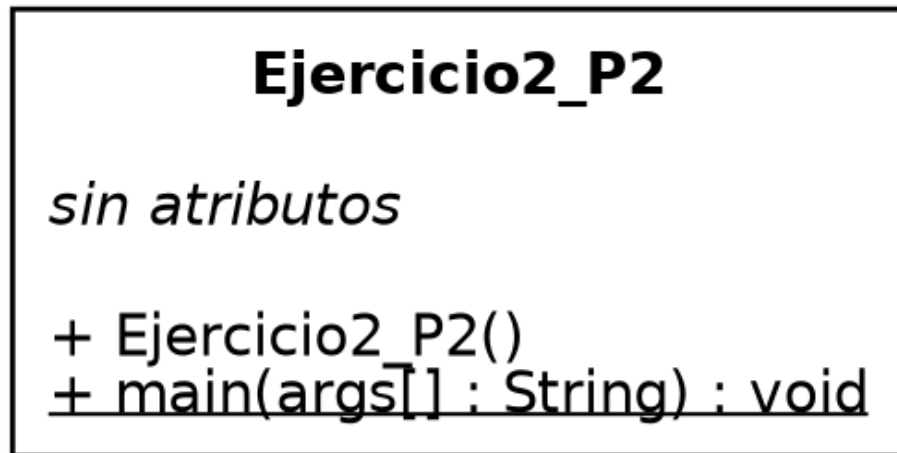


Figura 3.28: Diagrama UML de Ejercicio2_P2 (P2).

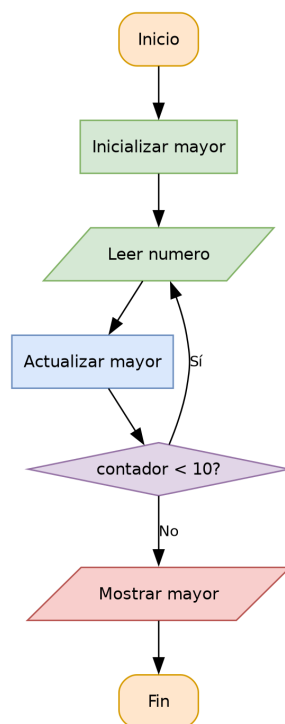


Figura 3.29: Diagrama de flujo de Ejercicio2_P2 (P2).

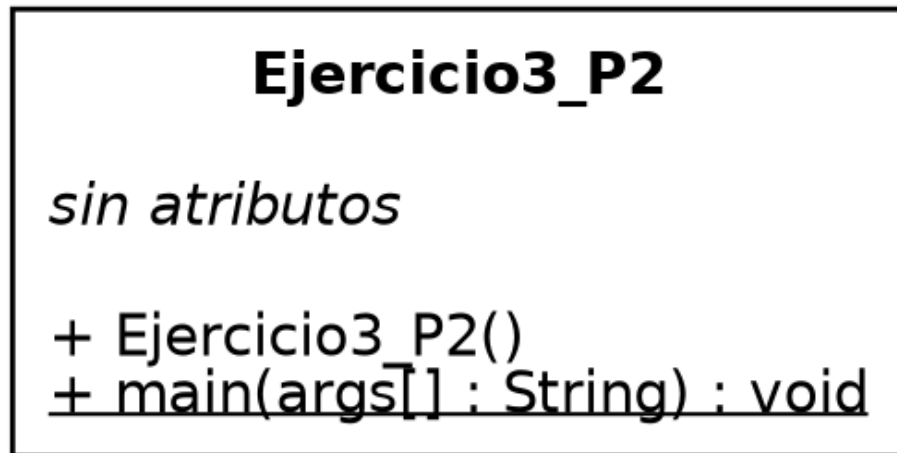


Figura 3.30: Diagrama UML de Ejercicio3_P2 (P2).

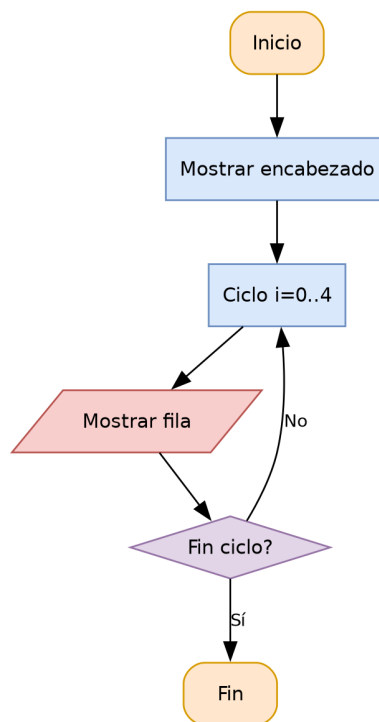


Figura 3.31: Diagrama de flujo de Ejercicio3_P2 (P2).

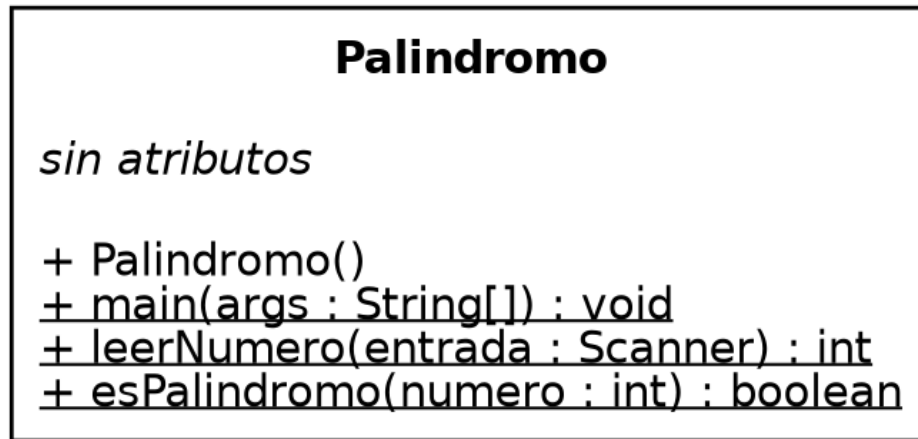


Figura 3.32: Diagrama UML de Palindromo (P3).

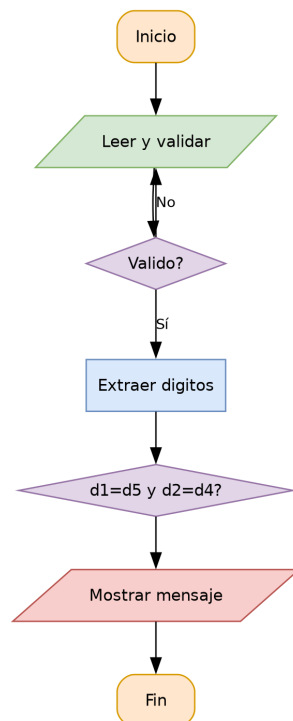


Figura 3.33: Diagrama de flujo de Palindromo (P3).

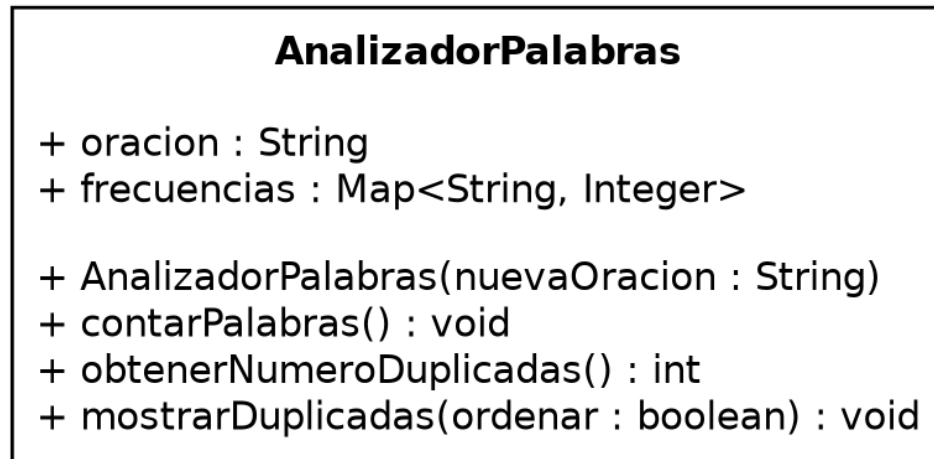


Figura 3.34: Diagrama UML de AnalizadorPalabras (P4).

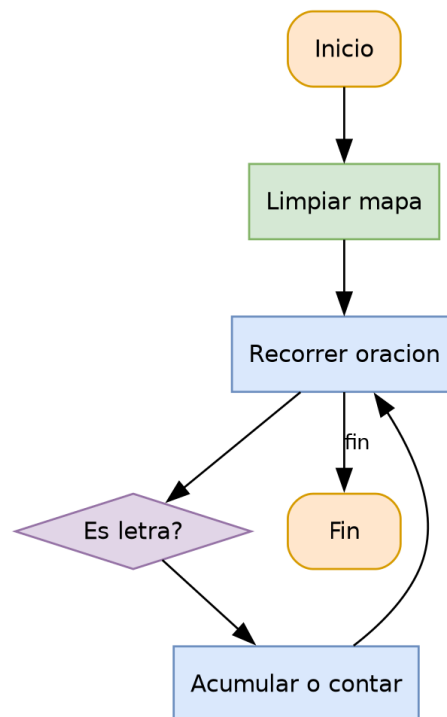


Figura 3.35: Diagrama de flujo de AnalizadorPalabras (P4).

3.4.16. Main de P4 (Análisis de palabras)

Coordina lectura y muestra duplicadas. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.36 se muestra el UML de **Main** y en la figura 3.37 su flujo.

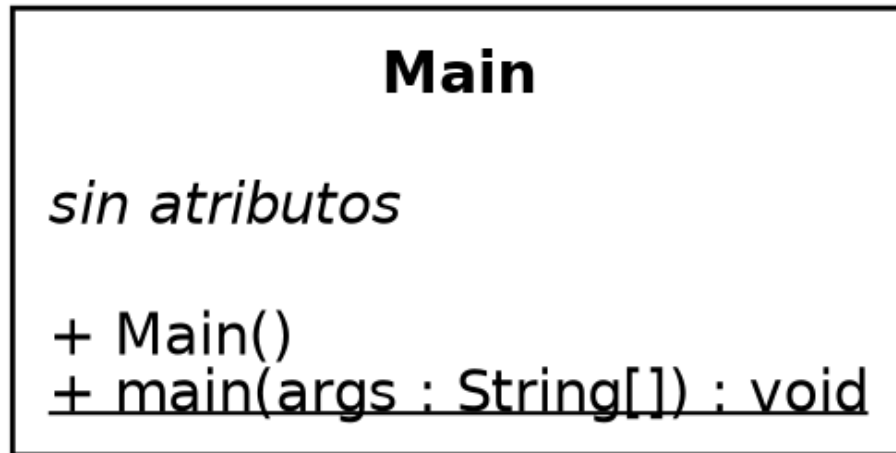


Figura 3.36: Diagrama UML de Main (P4).

3.4.17. Empleado de P5 (Encapsulamiento empleado y fecha)

Entidad empleado con salario validado. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.38 se muestra el UML de **Empleado** y en la figura 3.39 su flujo.

3.4.18. Fecha de P5 (Encapsulamiento empleado y fecha)

Entidad fecha con día mes y año. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.40 se muestra el UML de **Fecha** y en la figura 3.41 su flujo.

3.4.19. MainEmpleado de P5 (Encapsulamiento empleado y fecha)

Crea empleados y calcula anual. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.42 se muestra el UML de **MainEmpleado** y en la figura 3.43 su flujo.

3.4.20. MainFecha de P5 (Encapsulamiento empleado y fecha)

Crea fecha y la muestra. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.44 se muestra el UML de **MainFecha** y en la figura 3.45 su flujo.

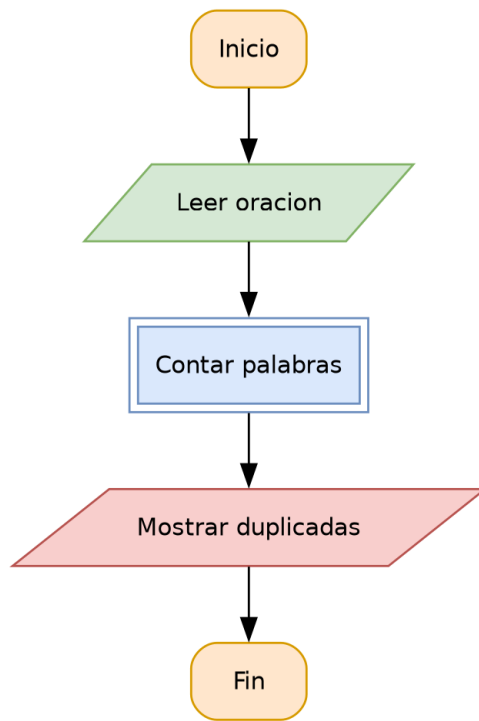


Figura 3.37: Diagrama de flujo de Main (P4).

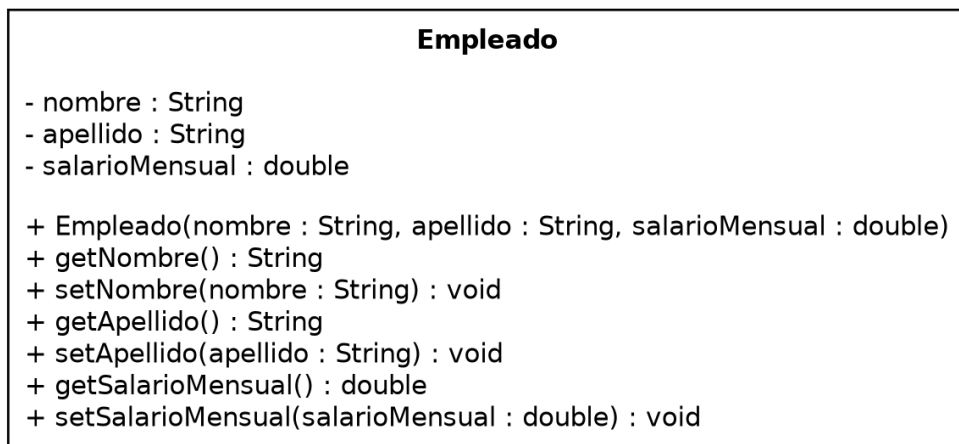


Figura 3.38: Diagrama UML de Empleado (P5).

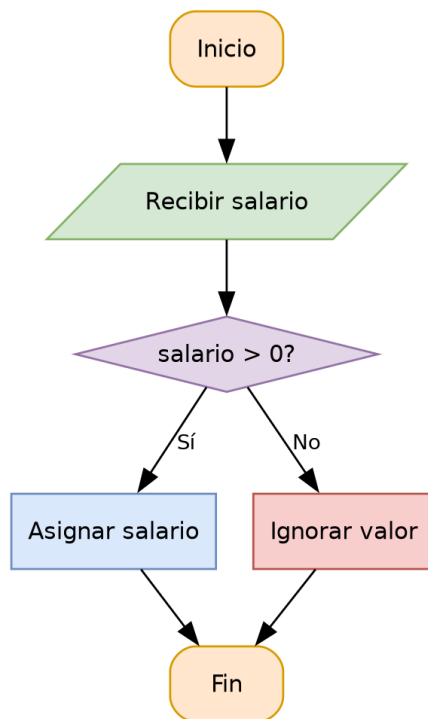


Figura 3.39: Diagrama de flujo de Empleado (P5).

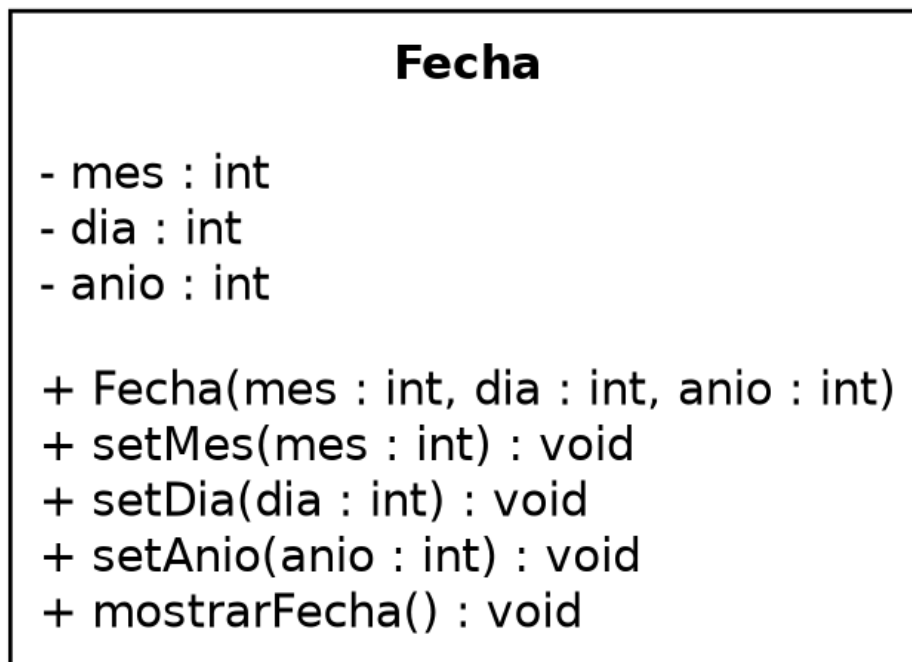


Figura 3.40: Diagrama UML de Fecha (P5).

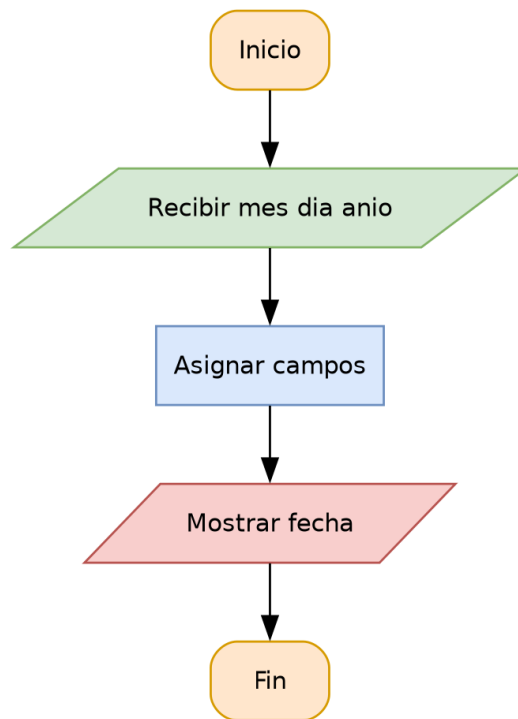


Figura 3.41: Diagrama de flujo de Fecha (P5).

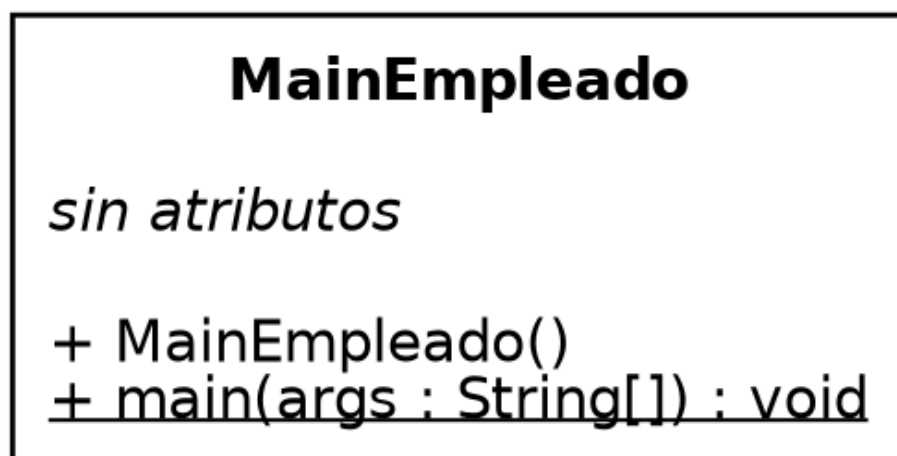


Figura 3.42: Diagrama UML de MainEmpleado (P5).

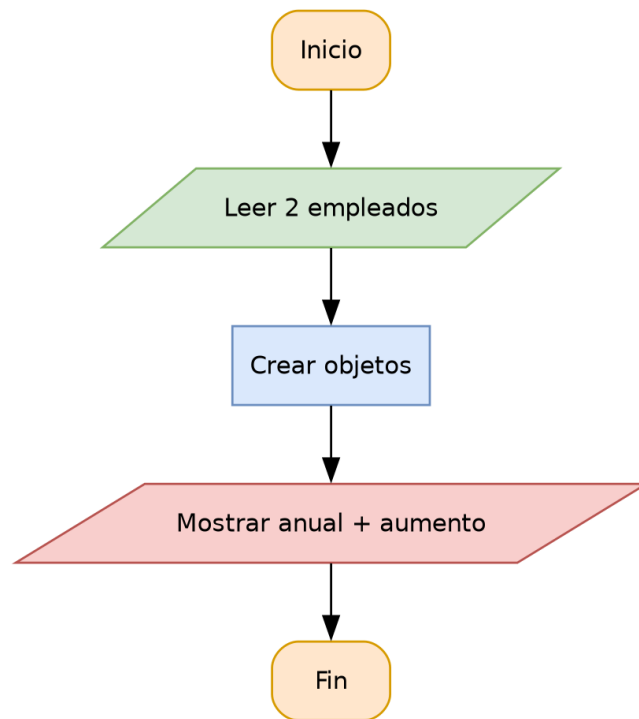


Figura 3.43: Diagrama de flujo de MainEmpleado (P5).

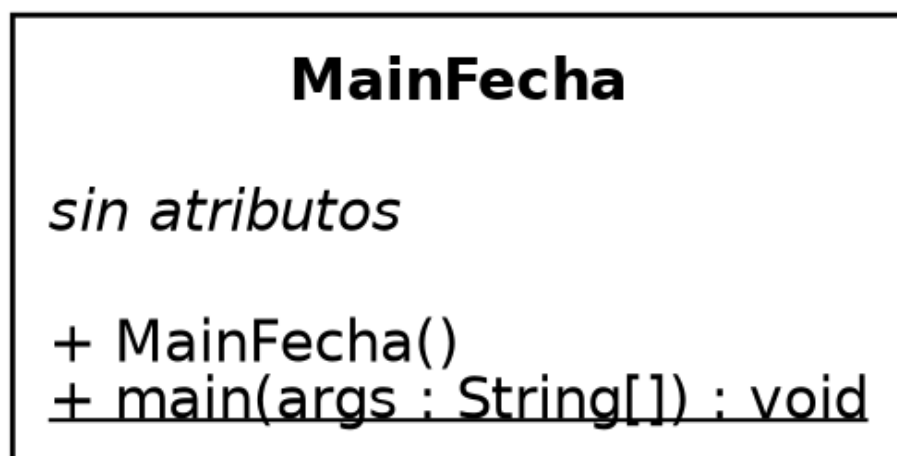


Figura 3.44: Diagrama UML de MainFecha (P5).

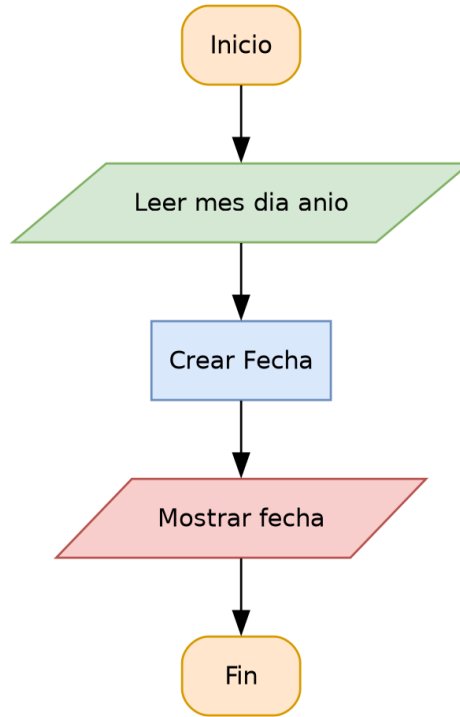


Figura 3.45: Diagrama de flujo de MainFecha (P5).

3.4.21. Medico de Proyecto (Sistema hospitalario)

Medico con especialidad y pacientes. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.46 se muestra el UML de Medico y en la figura 3.47 su flujo.

3.4.22. Enfermero de Proyecto (Sistema hospitalario)

Enfermero con atencion por especialidad. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.48 se muestra el UML de Enfermero y en la figura 3.49 su flujo.

3.4.23. Paciente de Proyecto (Sistema hospitalario)

Paciente con estado y asignaciones. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.50 se muestra el UML de Paciente y en la figura 3.51 su flujo.

3.4.24. Sistema de Proyecto (Sistema hospitalario)

Contenedor que registra personal y pacientes. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.52 se muestra el UML de Sistema y en la figura 3.53 su flujo.

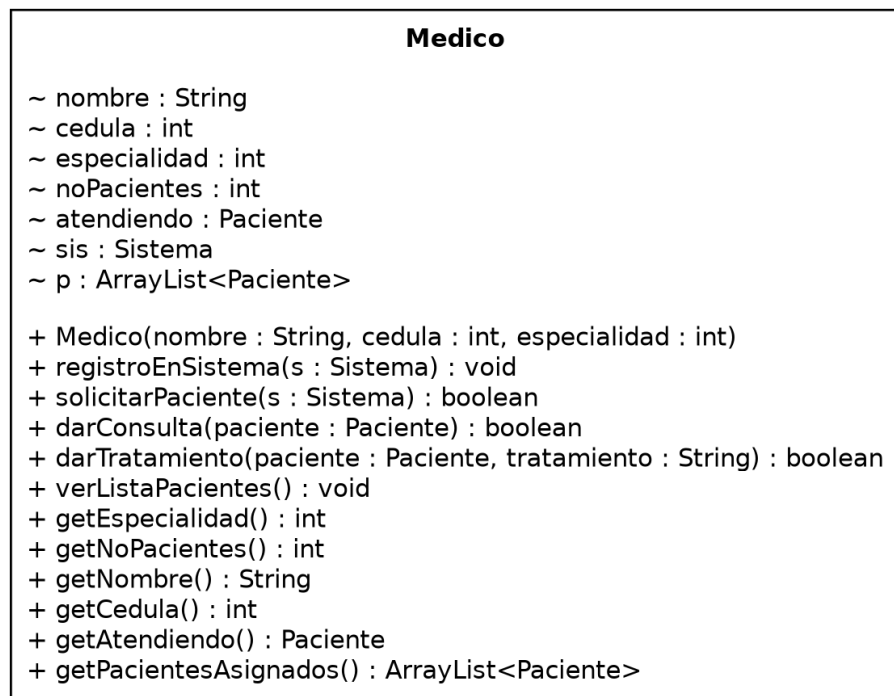


Figura 3.46: Diagrama UML de Medico (Proyecto).

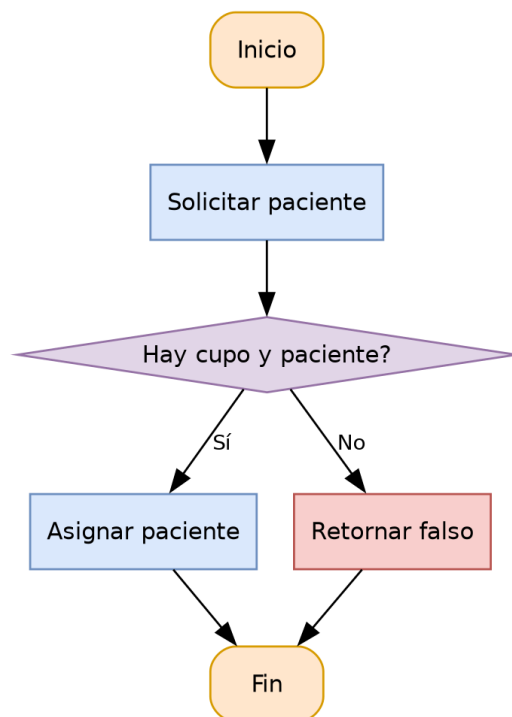


Figura 3.47: Diagrama de flujo de Medico (Proyecto).

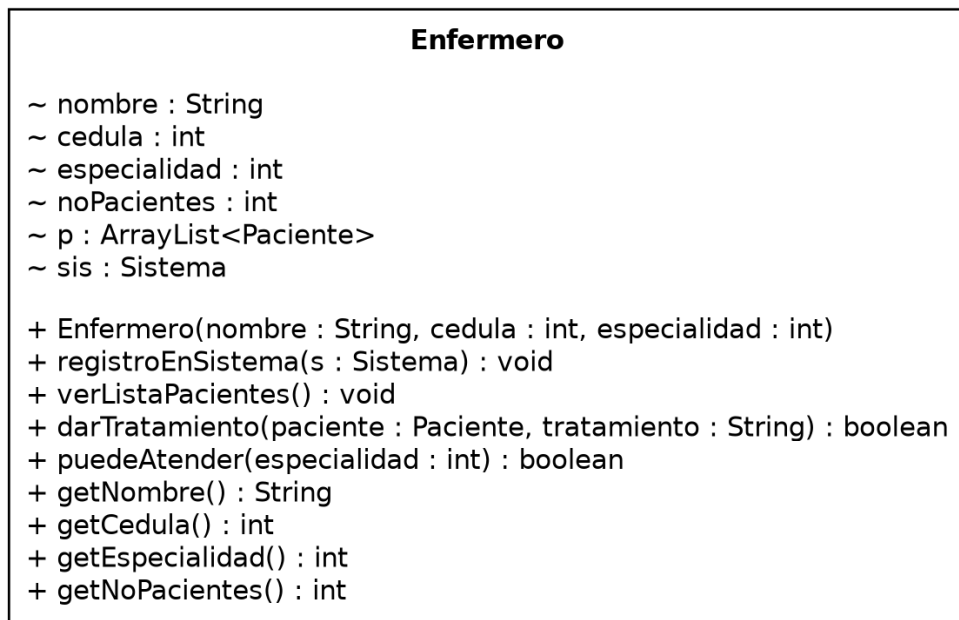


Figura 3.48: Diagrama UML de Enfermero (Proyecto).

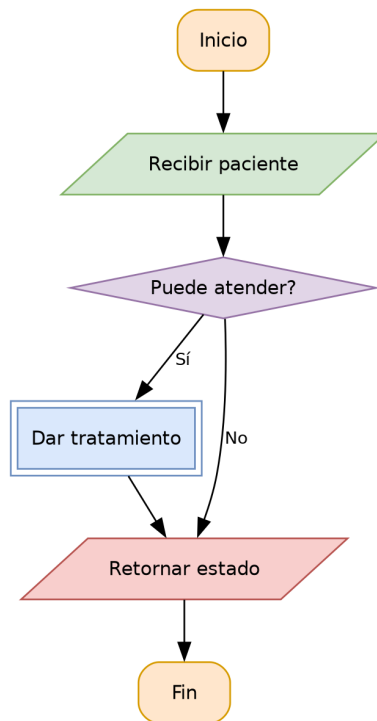


Figura 3.49: Diagrama de flujo de Enfermero (Proyecto).

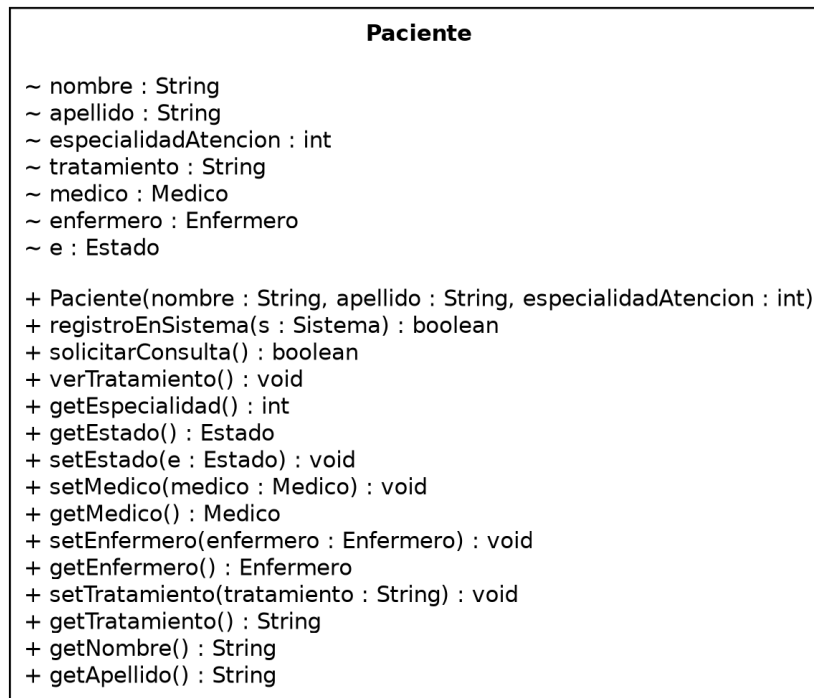


Figura 3.50: Diagrama UML de Paciente (Proyecto).

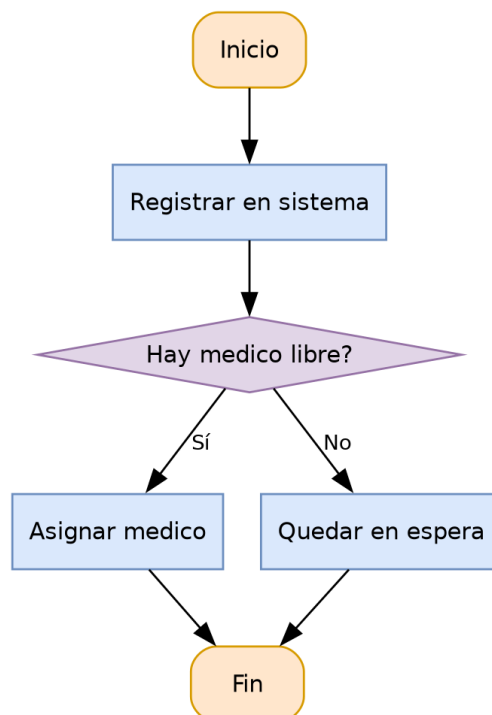


Figura 3.51: Diagrama de flujo de Paciente (Proyecto).

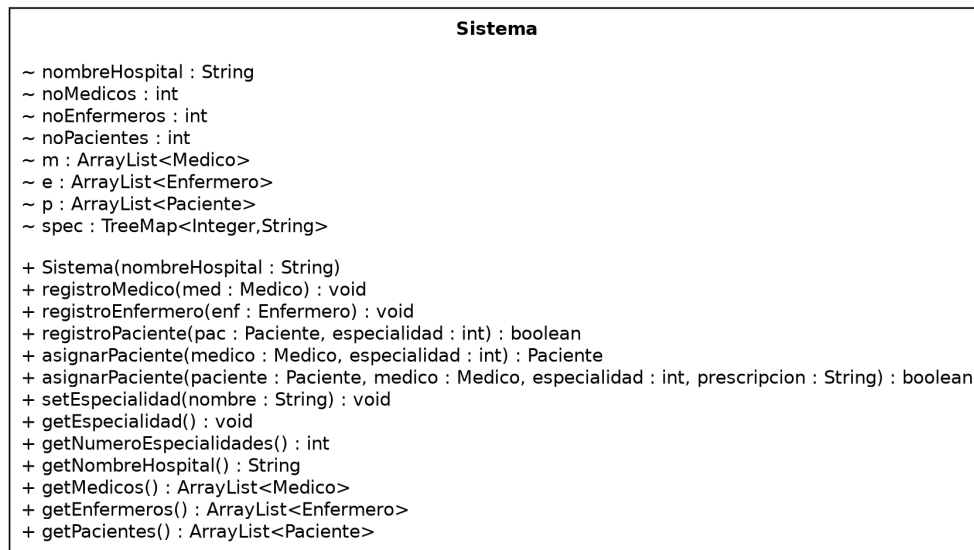


Figura 3.52: Diagrama UML de Sistema (Proyecto).

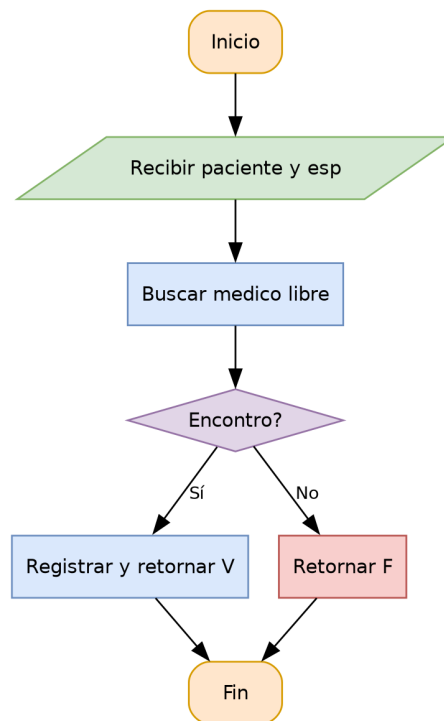


Figura 3.53: Diagrama de flujo de Sistema (Proyecto).

3.4.25. Main de Proyecto (Sistema hospitalario)

Menu principal del hospital. El UML resume atributos y operaciones verificados contra la compilación y el flujo detalla la decisión principal de la operación.

En la figura 3.54 se muestra el UML de Main y en la figura 3.55 su flujo.

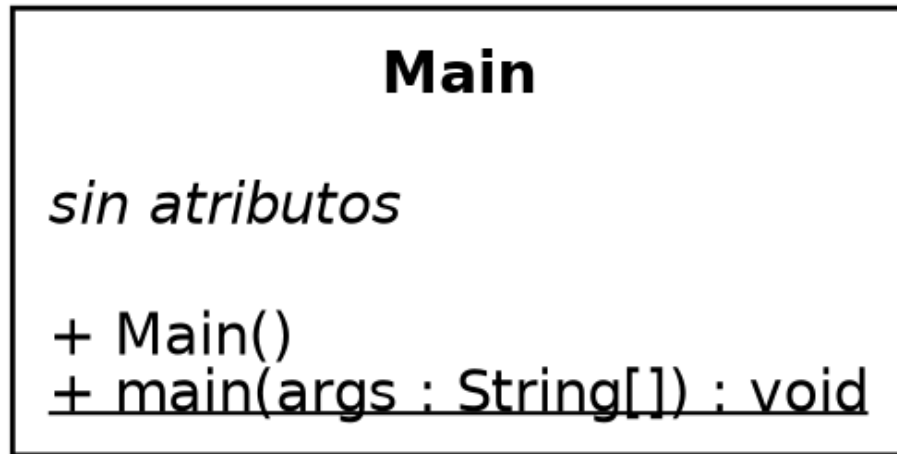


Figura 3.54: Diagrama UML de Main (Proyecto).

4. Resultados

4.1. Compilación de archivos empaquetados

Como entrada se tomaron los cinco archivos del paquete `mx.unam.fi.die.poo.g7.proyecto` ubicados en `src/`. El resultado esperado calculado consistía en la generación sin errores de la jerarquía `test/mx/.../proyecto/*.class` con codificación UTF-8. El resultado observado ejecutado coincidió: la invocación terminó sin salida de error y listó los binarios. En la figura 4.1 se muestra la captura real de la compilación.

4.2. Ejecución por classpath

Como entrada determinista se suministró por la entrada estándar la secuencia 1, 10 y 0: cargar datos demo, ver especialidades y salir. El resultado esperado calculado era el despliegue del menú, la confirmación de tres especialidades, tres médicos, tres enfermeros y tres pacientes, el listado de cardiología, pediatría y traumatología y el mensaje de salida. El resultado observado ejecutado con `java -cp test mx.unam.fi.die.poo.g7.proyecto.Main` coincidió de forma exacta. En la figura 4.2 se muestra la captura real.

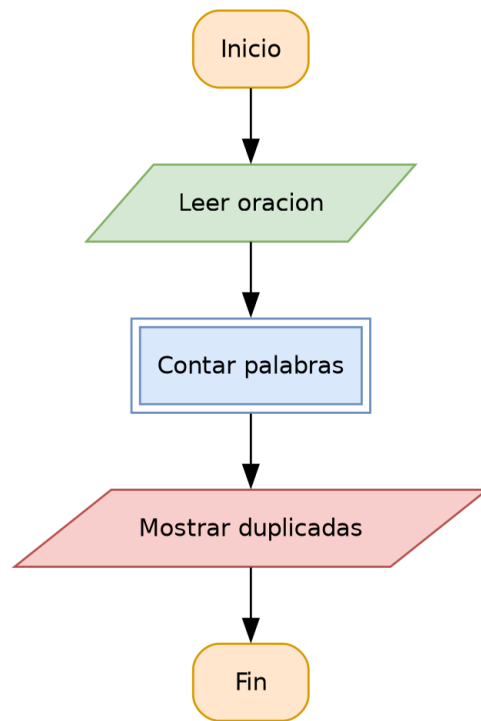


Figura 3.55: Diagrama de flujo de Main (Proyecto).

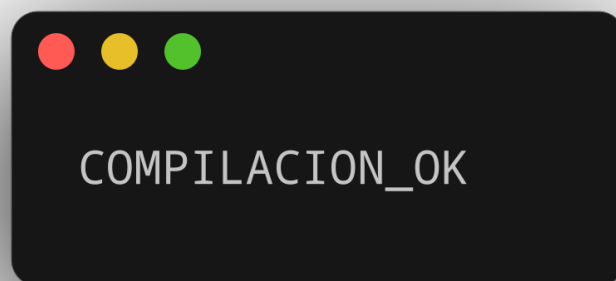


Figura 4.1: Compilación real de los archivos empaquetados del proyecto.

```
SISTEMA HOSPITALARIO
=====
(1) Cargar datos demo (3 de cada tipo)
=====
(2) Agregar hospital
(3) Agregar especialidad
(4) Agregar medico
(5) Agregar enfermero
(6) Agregar paciente
=====
(7) Medico solicita paciente
(8) Paciente solicita consulta
(9) Medico da tratamiento
=====
(10) Ver especialidades
(11) Ver medicos
(12) Ver enfermeros
(13) Ver pacientes
=====
(0) Salir
=====
Seleccione una opcion: Objetos demo cargados en: Hospital Central Universitario
3 Especialidades registradas
3 Medicos registrados
3 Enfermeros registrados
3 Pacientes en espera
=====
SISTEMA HOSPITALARIO
=====
(1) Cargar datos demo (3 de cada tipo)
=====
(2) Agregar hospital
(3) Agregar especialidad
(4) Agregar medico
(5) Agregar enfermero
(6) Agregar paciente
=====
(7) Medico solicita paciente
(8) Paciente solicita consulta
(9) Medico da tratamiento
=====
(10) Ver especialidades
(11) Ver medicos
(12) Ver enfermeros
(13) Ver pacientes
=====
(0) Salir
=====
Seleccione una opcion:
--- Especialidades del Hospital ---
1 - Cardiologia
2 - Pediatría
3 - Traumatología
=====
SISTEMA HOSPITALARIO
=====
(1) Cargar datos demo (3 de cada tipo)
=====
(2) Agregar hospital
(3) Agregar especialidad
(4) Agregar medico
(5) Agregar enfermero
(6) Agregar paciente
=====
(7) Medico solicita paciente
(8) Paciente solicita consulta
(9) Medico da tratamiento
=====
(10) Ver especialidades
(11) Ver medicos
(12) Ver enfermeros
(13) Ver pacientes
=====
(0) Salir
=====
Seleccione una opcion: Saliendo del sistema...
```

Figura 4.2: Ejecución real por classpath del sistema hospitalario.

4.3. Creación del JAR y verificación de contenido

Como entrada se usaron los binarios verificados de `test` y el manifiesto con `Main-Class: mx.unam.fi.die.poo.g7.proyecto.Main`. El resultado esperado calculado era un único archivo `lib/SistemaHospitalario.jar` con seis entradas de clases más manifiesto. El resultado observado con `jar cfm` y `jar tf` confirmó el contenido. En la figura 4.3 se muestra la captura real.

4.4. Ejecución del JAR

Como entrada se suministró `1, 11 y 0`: demo, ver médicos y salir. El resultado esperado era idéntico al de classpath pero invocando `java -jar lib/SistemaHospitalario.jar` sin declarar classpath. El resultado observado coincidió, con el listado de médicos y pacientes a cargo. En la figura 4.4 se muestra la captura real. La generación de Javadoc independiente por clase finalizó sin errores ni advertencias en `doc/Javadoc/P1/`, `P2/`, `P3/`, `P4/`, `P5/` y `Proyecto/`, con una carpeta por clase y navegación local. El JAR único en `lib/` contiene solo el proyecto verificado; `test/` conserva los 28 binarios del recopilatorio para classpath y documentación.

5. Conclusiones

La recopilación bajo paquetes institucionales eliminó la dispersión de entregas y resolvió colisiones aparentes como la de las dos clases `Main` mediante organización por

```
JAR_CREADO
META-INF/
META-INF/MANIFEST.MF
mx/unam/fi/die/poo/g7/proyecto/
mx/unam/fi/die/poo/g7/proyecto/Enfermero.class
mx/unam/fi/die/poo/g7/proyecto/Main.class
mx/unam/fi/die/poo/g7/proyecto/Medico.class
mx/unam/fi/die/poo/g7/proyecto/Paciente$Estado.class
mx/unam/fi/die/poo/g7/proyecto/Paciente.class
mx/unam/fi/die/poo/g7/proyecto/Sistema.class
```

Figura 4.3: Creación real del JAR y listado de su contenido.

```
===== SISTEMA HOSPITALARIO =====
(1) Cargar datos demo (3 de cada tipo)
=====
(2) Agregar hospital
(3) Agregar especialidad
(4) Agregar medico
(5) Agregar enfermero
(6) Agregar paciente
=====
(7) Medico solicita paciente
(8) Paciente solicita consulta
(9) Medico de tratamiento
=====
(10) Ver especialidades
(11) Ver medicos
(12) Ver enfermeros
(13) Ver pacientes
=====
(0) Salir
=====
Selección una opción: Objetos demo cargados en: Hospital Central Universitario
- 3 especialidades registradas
- 3 Medicos registrados
- 3 Enfermeros registrados
- 3 Pacientes en espera
=====
===== SISTEMA HOSPITALARIO =====
(1) Cargar datos demo (3 de cada tipo)
=====
(2) Agregar hospital
(3) Agregar especialidad
(4) Agregar medico
(5) Agregar enfermero
(6) Agregar paciente
=====
(7) Medico solicita paciente
(8) Paciente solicita consulta
(9) Medico de tratamiento
=====
(10) Ver especialidades
(11) Ver medicos
(12) Ver enfermeros
(13) Ver pacientes
=====
(0) Salir
=====
Selección una opción:
--- Medicos Registrados ---
Medico: Dr. Carlos Lopez | Codici: 101 | Exp: 1 | Pacientes: 0
Pacientes a cargo (0-2 por solicitud)
(Sin pacientes asignados)
Medico: Dra. Laura Gomez | Codici: 102 | Exp: 2 | Pacientes: 0
Pacientes a cargo (0-2 por solicitud)
(Sin pacientes asignados)
Medico: Dr. Juan Martinez | Codici: 103 | Exp: 3 | Pacientes: 0
Pacientes a cargo (0-2 por solicitud)
(Sin pacientes asignados)
=====
===== SISTEMA HOSPITALARIO =====
(1) Cargar datos demo (3 de cada tipo)
=====
(2) Agregar hospital
(3) Agregar especialidad
(4) Agregar medico
(5) Agregar enfermero
(6) Agregar paciente
=====
(7) Medico solicita paciente
(8) Paciente solicita consulta
(9) Medico de tratamiento
=====
(10) Ver especialidades
(11) Ver medicos
(12) Ver enfermeros
(13) Ver pacientes
=====
(0) Salir
=====
Selección una opción: Saliendo del sistema...
```

Figura 4.4: Ejecución real del JAR del sistema hospitalario.

origen, sin alterar la lógica original. La separación entre fuentes, binarios, biblioteca y documentación demostró ser la base mínima para portabilidad y mantenimiento en equipos heterogéneos.

Los comandos de compilación con destino explícito, ejecución por classpath y empaquetado con manifiesto forman una cadena reproducible: el mismo bytecode produce la misma salida ya sea desde `test` o desde el JAR. La documentación estandarizada y los diagramas uniformes permiten comparar clases de distinta dificultad con el mismo lenguaje visual y textual.

El ejemplo base del hospital validó la robustez del flujo con entradas deterministas de demo y consulta, confirmando que un diseño defensivo con validación de rangos y colecciones ordenadas sostiene la escalabilidad del sistema sin código adicional en esta práctica.

Referencias

- [1] P. J. Deitel y H. M. Deitel, *Cómo programar en Java*, 10.^a ed., Pearson Educación, México, 2016.
- [2] L. Joyanes Aguilar, *Fundamentos de programación: algoritmos, estructuras de datos y objetos*, 4.^a ed., McGraw-Hill Interamericana, Madrid, 2008.
- [3] Oracle Corporation, *The Java Tutorials: Packaging Programs in JAR Files*, Oracle Java SE Documentation, 2024.